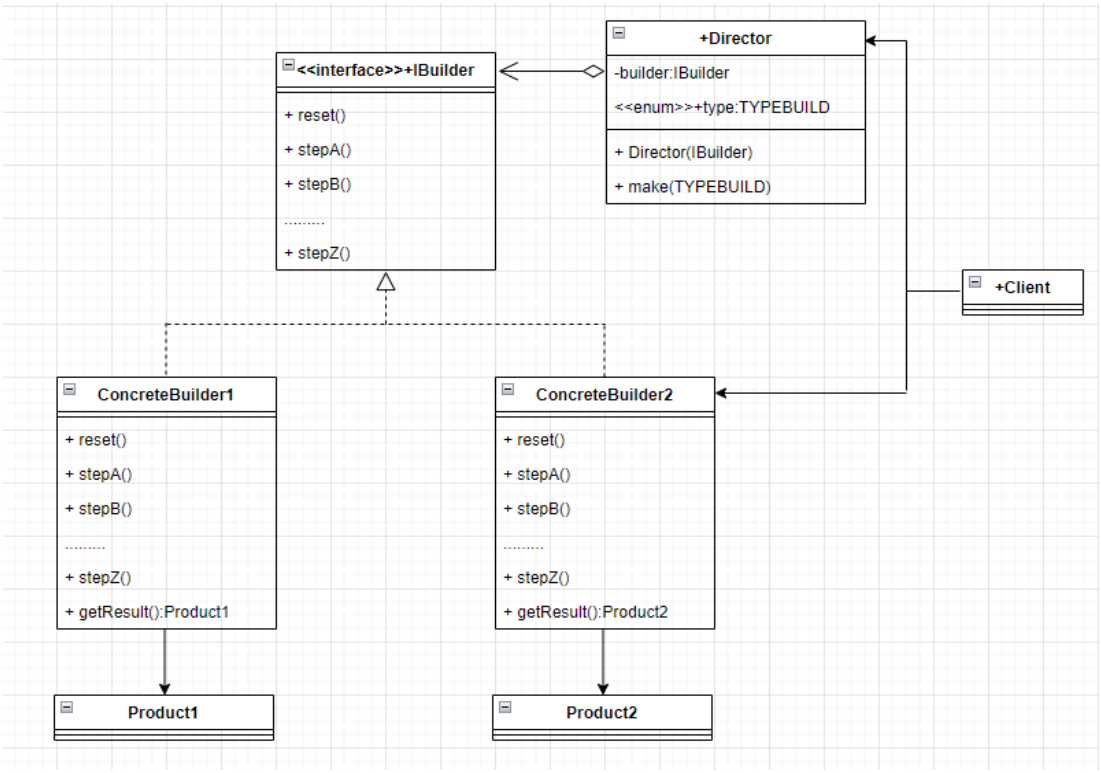
Порождающие паттерны(продолжение)

1. **Порождающие паттерны:** создание семейств новых объектов.
2. **Порождающие паттерны:** Factory Method (Фабричный метод), Abstract Factory (Абстрактная фабрика),**Builder** (Строитель), Prototype (Прототип), Singleton(Одиночка).

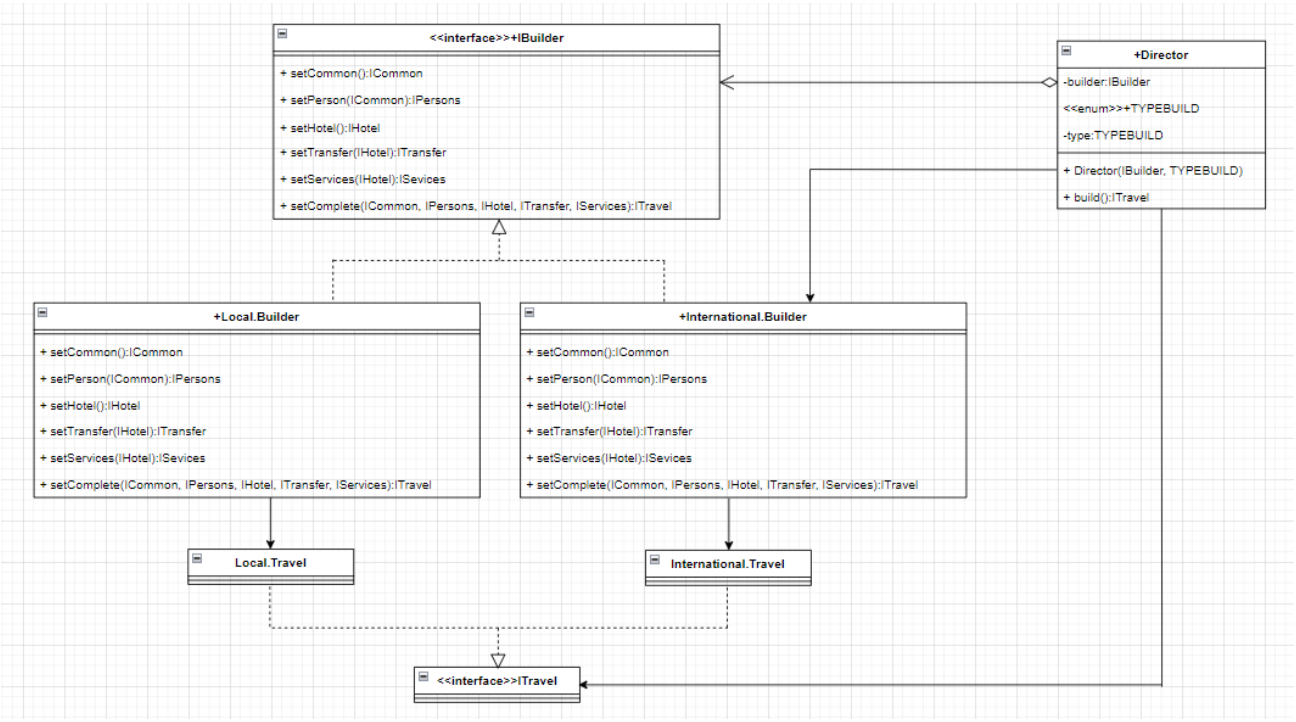
Builder

3. **Builder:** пошаговое создание сложных объектов.
4. **Описание проблемы:** объект (описание) создается пошагово (например, в результате диалога), шаги могут быть зависимыми.

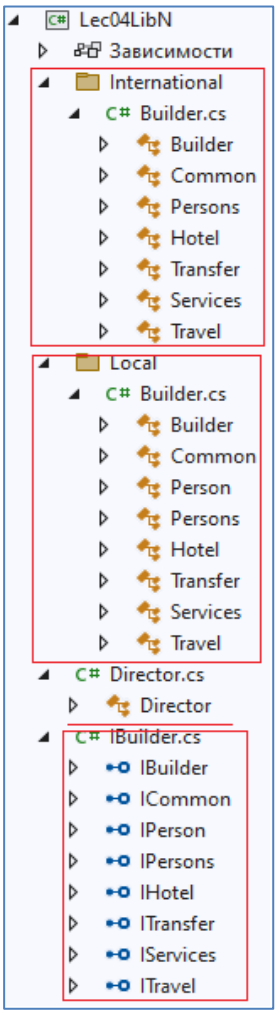
5. **UML-диаграмма: IBuilder –** интерфейс строителя, **ProductN** – построенные объекты (разные типы), **ConcreteBuilderN** – реализации строителя



6. Пример



7. C#




```
namespace Lec04LibN.Local
{
    //namespace Local
    //{

    public class Builder : IBuilder
    {
        public ICommon   setCommon()...
        public IPersons  setPersons(ICommon common)...
        public IHotel    setHotel(IPersons persons)...
        public ITransfer setTransfer(IHotel hotel)...
        public IServices setServices(IHotel hotel)...
        public ITravel   Complete(ICommon common, IHotel hotel, ITransfer transfer, IServices services)...
    }
    public class Common ...
        public class Person ...
        public class Persons ...
        public class Hotel ...
        public class Transfer ...
        public class Services ...
        public class Travel ...
    //}
}
```

```
namespace Lec04LibN.International
{
    public class Builder : IBuilder
    {
        public ICommon   setCommon()...
        public IPersons  setPersons(ICommon common)...
        public IHotel    setHotel(IPersons persons)...
        public ITransfer setTransfer(IHotel hotel)...
        public IServices setServices(IHotel hotel)...
        public ITravel   Complete(ICommon common, IHotel hotel, ITransfer transfer, IServices services)...
    }
    public class Common ...
    public class Persons ...
    public class Hotel ...
    public class Transfer ...
    public class Services ...
    public class Travel ...
}
```

```
namespace Lec04LibN
{
    public interface IBuilder
    {
        ICommon   setCommon();
        IPersons  setPersons(ICommon common);
        IHotel    setHotel(IPersons persons);
        ITransfer setTransfer(IHotel hotel);
        IServices setServices(IHotel hotel);
        ITravel   Complete(ICommon common, IHotel hotel, ITransfer transfer, IServices services);
    }


    public interface ICommon...
    public interface IPerson...
    public interface IPersons...
    public interface IHotel...
    public interface ITransfer...
    public interface IServices...
    public interface ITravel...
}
```

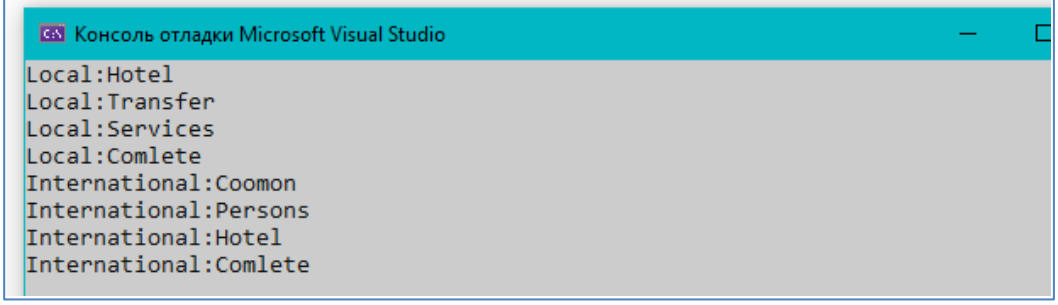
```
public class Director
{
    IBuilder builder;
    ICommon common      { get; set; }
    IPersons persons    { get; set; }
    IHotel hotel        { get; set; }
    ITransfer transfer  { get; set; }
    IServices services  { get; set; }
    ITravel travel      { get; set; }
    const int COMMON = 1, PERSONS = 2, HOTEL = 4, TRANSFER = 8, SERVICES = 16, TRAVEL = 32;
    const int REGIM_S = COMMON | PERSONS | HOTEL | TRANSFER | SERVICES | TRAVEL,
                        REGIM_A = COMMON | PERSONS | HOTEL | TRANSFER | TRAVEL,
                        REGIM_B = COMMON | PERSONS | HOTEL | TRAVEL;
    public enum TYPEREGIM { S = REGIM_S, A = REGIM_A, B = REGIM_B }
    TYPEREGIM regim = TYPEREGIM.S;
    public Director(IBuilder builder)
    {
        this.builder = builder;
        this.regim = TYPEREGIM.S;
    }
    public Director(IBuilder builder, TYPEREGIM regim)
    {
        this.builder = builder;
        this.regim = regim;
    }
    public bool build()
    {

        bool rc = true;
        int sreg = (int)this.regim;
        if ((sreg & COMMON) == COMMON)
        {
            common = builder.setCommon();
        }
        if ((sreg & PERSONS) == PERSONS)
        {
            persons = builder.setPersons(common);
        }
        if ((sreg & HOTEL) == HOTEL)
        {
            hotel = builder.setHotel(persons);
        }
        if ((sreg & TRANSFER) == TRANSFER)
        {
            transfer = builder.setTransfer(hotel);
        }
        if ((sreg & SERVICES) == SERVICES)
        {
            services = builder.setServices(hotel);
        }
        if ((sreg & TRAVEL) == TRAVEL)
        {
            travel = builder.Complete(common,hotel,transfer,services);
        }
        return rc;
    }
    public ITravel getResult()
    {
        return this.travel;
    }
}
```

```
static void Main(string[] args)
{
    IBuilder l_builder  =  new Local.Builder();
    Director l_director = new Director(l_builder, Director.TYPEREGIM.S);
    if (l_director.build())
    {
        ITravel travel = l_director.getResult();
    }

    IBuilder i_builder  = new International.Builder();
    Director i_director = new Director(i_builder, Director.TYPEREGIM.B);
    if (i_director.build())
    {
        ITravel travel = i_director.getResult();
    }

}
```



8. **КОНЕЦ**

```
public class Director
{
    IBuilder builder;
    ICommon common     { get; set; }
    IPersons persons   { get; set; }
    IHotel hotel       { get; set; }
    ITransfer transfer { get; set; }
    IServices services { get; set; }
    ITravel travel     { get; set; }
    const int COMMON = 1, PERSONS = 2, HOTEL = 4, TRANSFER = 8, SERVICES = 16, TRAVEL = 32;
    const int REGIM_S = COMMON | PERSONS | HOTEL | TRANSFER | SERVICES | TRAVEL,
                        REGIM_A = COMMON | PERSONS | HOTEL | TRANSFER | TRAVEL,
                        REGIM_B = COMMON | PERSONS | HOTEL | TRAVEL;
    public enum TYPEREGIM { S = REGIM_S, A = REGIM_A, B = REGIM_B }
    TYPEREGIM regim = TYPEREGIM.S;
    public Director(IBuilder builder)
    {
        this.builder = builder;
        this.regim = TYPEREGIM.S;
    }
    public Director(IBuilder builder, TYPEREGIM regim)
    {
        this.builder = builder;
        this.regim = regim;
    }
    public bool build()
    {

        bool rc = true;
        int sreg = (int)this.regim;
        if ((sreg & COMMON) == COMMON)
        {
            common = builder.setCommon();
        }
        if ((sreg & PERSONS) == PERSONS)
        {
            persons = builder.setPersons(common);
        }
        if ((sreg & HOTEL) == HOTEL)
        {
            hotel = builder.setHotel(persons);
        }
        if ((sreg & TRANSFER) == TRANSFER)
        {
            transfer = builder.setTransfer(hotel);
        }
        if ((sreg & SERVICES) == SERVICES)
        {
            services = builder.setServices(hotel);
        }
        if ((sreg & TRAVEL) == TRAVEL)
        {
            travel = builder.Complete(common,hotel,transfer,services);
        }
        return rc;
    }
    public ITravel getResult()
    {
     return this.travel;
    }
}
```